INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

APLICACIONES PARA COMUNICACIONES EN RED

Reporte

“Servidor HTTP parte 2”

Grupo: 6CM2

Integrantes

- García Escamilla Bryan Alexis
- Meléndez Macedonio Rodrigo

Profesor

Moreno Cervantes Axel Ernesto

Fecha de entrega: 06 de enero de 2025

NTRODUCCIÓN

Socket no bloqueante

Un socket no bloqueante es un socket configurado para que nunca detenga la ejecución del programa mientras espera que ocurra una operación de red, como: recibir datos, enviar datos y establecer comunicación.

En lugar de esperar a que la operación se complete, el socket responde de inmediato, aunque no haya datos disponibles en ese momento. Si no puede realizar la operación ahora mismo, regresa inmediatamente indicando que aún no es posible.

Socket no bloqueante	Socket bloqueante
• El programa pregunta si puede hacer la operación. • Si no puede, continúa ejecutándose y vuelve a intentarlo después. • Ideal para programas interactivos o concurrentes.	El programa se queda detenido esperando: • A que lleguen datos • A que haya espacio para enviar Es simple, pero puede hacer que el programa se congele.

Cuando se usa un socket no bloqueante:

- Si intenta recibir datos y no hay nada: el sistema no espera y devuelve un aviso de que no hay datos disponibles.
- Si intenta enviar datos y el buffer está lleno: no se bloquea e indica que no puede enviar todavía

El control del tiempo y los reintentos queda en manos del programa.

Sin embargo, los sockets no bloqueantes son más complejos de implementar, ya que requieren manejar; estados, errores temporales y reintentos. Y su mal uso pueden causar bucles infinitos o mal consumo de CPU, por eso suelen combinarse con mecanismos que esperen eventos de forma eficiente.

Biblioteca “selectors” en Python

`selectors` es una biblioteca estándar de Python que no necesita instalar nada, la cual proporciona una forma portable y de alto nivel para vigilar múltiples objetos de entrada y salida como sockets y saber cuándo están listos para leer o escribir, sin bloquear el programa.

Característica	Descripción
No bloquea	Evita esperas innecesarias.
Eficiente	Usa mecanismos nativos del SO.
Portable	Funciona igual en Windows, Linux y macOS.
Escalable	Maneja muchas conexiones.
Alto nivel	Más simple que usar APIs del SO directamente.

Funciones principales de la biblioteca "selectors"

Función	Descripción
`selectors.DefaultSelector()`	Crea un selector usando el mejor mecanismo disponible en tu sistema operativo.
`register(fileobj, events, data=None)`	Registra un objeto de entrada/salida (como un socket) para que el selector lo vigile.
`unregister(fileobj)`	Elimina un socket del selector.
`modify(fileobj, events, data=None)`	Cambia los eventos que te interesan para un socket ya registrado.
`select(timeout=None)`	Espera a que un socket este listo o se cumple el tiempo límite.
`close()`	Cierra el selector y libera recursos.

Eventos de la biblioteca "selectors"

Evento	¿Qué indica?
`EVENT_READ`	El socket está listo para leer.
`EVENT_WRITE`	El socket está listo para escribir.

Flujo típico de uso

Crear selector → Registrar sockets → Esperar eventos → Procesar sockets listos → Modificar o eliminar registros → Cerrar selector

DESARROLLO

Para el servidor HTTP solo se creó un solo archivo, servidor.py.

```python
HOST = "127.0.0.1"

PORT_PRIMARIO = 8080
PORT_SECUNDARIO = 8081

POOL_MAX = 3
MITAD_POOL = POOL_MAX // 2

BASE_DIR = Path(__file__).parent
ARCHIVES = BASE_DIR / "archives"

# selector principal que monitorea todos los sockets
selector = selectors.DefaultSelector()

conexiones_activas = {
    PORT_PRIMARIO: 0,
    PORT_SECUNDARIO: 0
}

MIME_TYPES = {
    ".html": "text/html",
    ".txt": "text/plain",
    ".json": "application/json",
    ".png": "image/png",
    ".jpg": "image/jpeg",
    ".jpeg": "image/jpeg",
    ".mp4": "video/mp4",
    ".mp3": "audio/mpeg",
    ".pdf": "application/pdf",
}

# se indican los tipos que el navegador puede mostrar
MIMES_RENDERIZABLES = set(MIME_TYPES.values())
```

Imagen 1. Variables globales.

Aquí se declaran las variables necesarias para el funcionamiento del programa. Se define la dirección IP del servidor y los puertos para ambos servidores, así como el pool de conexiones (en este caso el pool de conexiones es muy pequeño para observar la redirección). En este caso ambos servidores usan la misma carpeta en donde se encuentran los recursos.

La novedad en este programa es que ahora tenemos un selector, este es el responsable de observar muchos sockets, se encarga de avisarle al servidor cuando alguno de ellos puede leer o escribir.

También tenemos un diccionario que actualiza el número de conexiones activas en ambos servidores.

Y por último tenemos los tipos de MIME, que en este caso aquellos que se encuentran en el diccionario serán los únicos que el navegador podrá mostrar, mientras que aquellos que no están serán forzados a descargarse. En el caso de que el recurso sea una carpeta, su contenido se comprimirá en un archivo ZIP y se descargará.

- **log_peticion(puerto: int, linea: str)**

```
44  def log_peticion(puerto: int, linea: str) -> None:
45      print(f"[PETICIÓN] ({puerto}) {linea}")
```

Imagen 2. Función log_peticion(puerto: int, linea: str).

Descripción general: Se encarga de mostrar en consola la petición recibida desde el navegador.

Parámetros:

- **puerto : int**
  - Puerto donde llegó la petición.
- **linea : str**
  - Primera línea del request HTTP.

- **log_respuesta(puerto: int, status: str)**

```
58  def log_respuesta(puerto: int, status: str) -> None:
59      print(f"[RESPUESTA] ({puerto}) {status}")
```

Imagen 3. Función log_respuesta(puerto: int, status: str).

Descripción general: Se encarga de mostrar la respuesta enviada por el servidor.

Parámetros:

- **puerto : int**
  - Puerto que envía la respuesta.
- **status : str**
  - Código HTTP enviado.

- class Conexión

```python
class Conexion:
    def __init__(self, sock: socket.socket, puerto: int):
        self.sock = sock
        self.puerto = puerto
        self.inb = b"" # buffer de entrada (bytes recibidos)
        self.outb = b"" # buffer de salida (respuesta)
        self.headers_completos = False
        self.content_length = 0 # tamaño del body esperado
```

Imagen 4. Clase class Conexión.

Descripción general: Clase que representa una conexión.

- **aceptar(sock: socket.socket, puerto: int)**

```python
def aceptar(sock: socket.socket, puerto: int) -> None:
    cliente, _ = sock.accept()
    cliente.setblocking(False)

    if conexiones_activas[puerto] >= POOL_MAX: # si el pool está lleno
        cliente.sendall(
            b"HTTP/1.1 503 Service Unavailable\r\n\r\nServidor saturado"
        )
        cliente.close()
        return

    conexiones_activas[puerto] += 1 # incrementa el pool solo si entra
    selector.register(cliente, selectors.EVENT_READ, data=Conexion(cliente, puerto))
```

Imagen 5. Función aceptar(sock: socket.socket, puerto: int).

Descripción general: Se encarga de aceptar una conexión entrante, verificar si el pool de conexiones está lleno, registrar el socket cliente en el selector e incrementar el contador de conexiones.

Parámetros:

- **sock : socket.socket**
  - Socket del servidor.
- **puerto : int**
  - Puerto donde llegó la conexión.

- **leer(con: Conexion)**

```python
def leer(con: Conexion) -> None:
    datos = con.sock.recv(4096)
    if not datos:
        cerrar(con)
        return

    con.inb += datos

    # headers
    if not con.headers_completos and b"\r\n\r\n" in con.inb:
        headers, resto = con.inb.split(b"\r\n\r\n", 1)
        con.headers_completos = True

        for linea in headers.decode(errors="ignore").splitlines():
            if linea.lower().startswith("content-length"):
                con.content_length = int(linea.split(":")[1].strip())

        con.inb = headers + b"\r\n\r\n" + resto

    # se verifica si ya esta todo
    if con.headers_completos:
        total = len(con.inb.split(b"\r\n\r\n", 1)[1])
        if total >= con.content_length:
            procesar_peticion(con)
```

Imagen 6. Función leer(con: Conexión).

Descripción general: Se encarga de leer los datos del socket sin bloquear, de acumular bytes en el buffer de entrada, detectar el final de los headers.

Parámetros:

- **con : Conexion**
  - Objeto de conexión activa

- **procesar_peticion(con: Conexion)**

```python
def procesar_peticion(con: Conexion) -> None:
    texto = con.inb.decode(errors="ignore")
    lineas = texto.splitlines()
    linea = lineas[0]
    log_peticion(con.puerto, linea)

    try:
        metodo, ruta, _ = linea.split()
    except ValueError:
        enviar_error(con, 400, "Bad Request")
        return

    # redirige las peticiones al segundo servidor (302)
    if con.puerto == PORT_PRIMARIO and conexiones_activas[PORT_PRIMARIO] > MITAD_POOL:
        redirigir(con, PORT_SECUNDARIO)
        return

    # resolver la ruta principal
    if ruta == "/":
        ruta = "/index.html"

    path = ARCHIVES / ruta.lstrip("/")

    body = texto.split("\r\n\r\n", 1)[1].encode()
```

Imagen 7. Función procesar_peticion(con: Conexion).

Descripción general: Se encarga de extraer el método, ruta y versión de la petición, registrar la petición, validar los métodos GET, POST, PUT y DELETE, así como de balancear la carga, redirigiendo las peticiones al segundo servidor al superar el pool de conexiones.

Parámetros:

- **con : Conexion**
    - Objeto de conexión activa

```python
    # <====== GET ======>
    if metodo == "GET":
        if path.exists() and path.is_dir():
            if not ruta.endswith("/"):
                redirigir_ruta(con, ruta + "/") # 301
                return
            enviar_zip(con, path)
            return

        if not path.exists() or not path.is_file():
            enviar_error(con, 404, "Not Found")
            return

        mime = MIME_TYPES.get(path.suffix, "application/octet-stream")
        contenido = path.read_bytes()

        headers = [
            "HTTP/1.1 200 OK",
            f"Content-Type: {mime}",
            f"Content-Length: {len(contenido)}",
        ]

        if mime not in MIMES_RENDERIZABLES:
            headers.append(
                f'Content-Disposition: attachment; filename="{path.name}"'
            )

        headers.append("\r\n")
        con.outb = ("\r\n".join(headers)).encode() + contenido
        log_respuesta(con.puerto, "200 OK")
```

Imagen 8. Procesar método GET.

Descripción general: Obtiene la ruta del recurso y si es necesario la redirige en caso de que no termine en '/'. Si el recurso coincide con alguno de los tipos de MIME el navegador lo mostrará, de lo contrario este será forzado a descargarse. Y, por último, si el recurso no se encuentra se envía el código 404, de lo contrario se envía 200.

```python
    # <====== POST ======>
    elif metodo == "POST":
        if path.exists():
            enviar_error(con, 409, "Conflict") # si ya existe
            return

        path.parent.mkdir(parents=True, exist_ok=True)
        path.write_bytes(body)

        respuesta = b"Archivo creado correctamente"
        con.outb = (
            "HTTP/1.1 201 Created\r\n"
            "Content-Type: text/plain\r\n"
            f"Content-Length: {len(respuesta)}\r\n\r\n"
        ).encode() + respuesta
        log_respuesta(con.puerto, "201 Created")
```

Imagen 9. Procesar método POST.

Descripción general: Si la ruta del recurso no existe, se crea y envía el código 201. Si el archivo ya ha sido creado entonces envía 409.

```python
    # <====== PUT ======>
    elif metodo == "PUT":
        path.parent.mkdir(parents=True, exist_ok=True)
        path.write_bytes(body)

        respuesta = b"Archivo creado/actualizado"
        con.outb = (
            "HTTP/1.1 200 OK\r\n"
            "Content-Type: text/plain\r\n"
            f"Content-Length: {len(respuesta)}\r\n\r\n"
        ).encode() + respuesta
        log_respuesta(con.puerto, "200 OK")
```

Imagen 10. Procesar método PUT.

Descripción general: Si la ruta del archivo no existe, se crea/modifica y envía el código 200.

```python
    # <====== DELETE ======>
    elif metodo == "DELETE":
        if not path.exists():
            enviar_error(con, 404, "Not Found") # no existe
            return

        path.unlink()
        respuesta = b"Archivo eliminado"
        con.outb = (
            "HTTP/1.1 200 OK\r\n"
            "Content-Type: text/plain\r\n"
            f"Content-Length: {len(respuesta)}\r\n\r\n"
        ).encode() + respuesta
        log_respuesta(con.puerto, "200 OK")

    else:
        enviar_error(con, 405, "Method Not Allowed")
        return

    selector.modify(con.sock, selectors.EVENT_WRITE, data=con)
```

Imagen 11. Procesar método DELETE.

Descripción general: Si el archivo no existe entonces se envía el código 404, si este existe se elimina y envía 200.

- **enviar_zip(con: Conexion, carpeta: Path)**

```python
def enviar_zip(con: Conexion, carpeta: Path) -> None:
    buffer = io.BytesIO()
    with zipfile.ZipFile(buffer, "w", zipfile.ZIP_DEFLATED) as zipf:
        for archivo in carpeta.rglob("*"):
            if archivo.is_file():
                zipf.write(archivo, archivo.relative_to(carpeta))

    data = buffer.getvalue()
    con.outb = (
        "HTTP/1.1 200 OK\r\n"
        "Content-Type: application/zip\r\n"
        f"Content-Length: {len(data)}\r\n"
        f'Content-Disposition: attachment; filename="{carpeta.name}.zip"\r\n\r\n'
    ).encode() + data
    log_respuesta(con.puerto, "200 OK (ZIP)")
    selector.modify(con.sock, selectors.EVENT_WRITE, data=con)
```

Imagen 12. Función enviar_zip(con: Conexion, carpeta: Path).

Descripción general: Se encarga de comprimir la carpeta en memoria, conservando la estructura interna y envía ZIP como descarga.

Parámetros:

- **con : Conexion**
  - Objeto de conexión activa
- **carpeta : Path**
  - Ruta de la carpeta de los recursos

- **redirigir(con: Conexion, puerto_destino: int)**

```
285 def redirigir(con: Conexion, puerto_destino: int) -> None:
286     con.outb = (
287         "HTTP/1.1 302 Found\r\n"
288         f"Location: http://{HOST}:{puerto_destino}\r\n\r\n"
289     ).encode()
290     log_respuesta(con.puerto, f"302 Redirect -> {puerto_destino}")
291     selector.modify(con.sock, selectors.EVENT_WRITE, data=con)
```

Imagen 13. Función redirigir(con: Conexion, puerto_destino: int).

Descripción general: Se encarga de redirigir la petición al segundo servidor y de envíar el código 302 a este.

Parámetros:

- **con : Conexión**
  - Objeto de conexión activa
- **puerto_destino : int**
  - Puerto del segundo servidor

- **redirigir_ruta(con: Conexion, nueva: str)**

```
304 def redirigir_ruta(con: Conexion, nueva: str) -> None:
305     con.outb = (
306         "HTTP/1.1 301 Moved Permanently\r\n"
307         f"Location: {nueva}\r\n\r\n"
308     ).encode()
309     log_respuesta(con.puerto, "301 Moved Permanently")
310     selector.modify(con.sock, selectors.EVENT_WRITE, data=con)
```

Imagen 14. Función redirigir_ruta(con: Conexion, nueva: str).

Descripción general: Se encarga de redirigir la ruta permanentemente si esta no termina en '/' enviando el código 301.

Parámetros:

- **con : Conexion**

- Objeto de conexión activa.

- **nueva : str**
  - Ruta de la carpeta con / al final

- **enviar_error(con: Conexion, codigo: int, mensaje: str)**

```
324 def enviar_error(con: Conexion, codigo: int, mensaje: str) -> None:
325     body = f"<h1>{codigo} {mensaje}</h1>".encode()
326     con.outb = (
327         f"HTTP/1.1 {codigo} {mensaje}\r\n"
328         "Content-Type: text/html\r\n"
329         f"Content-Length: {len(body)}\r\n\r\n"
330     ).encode() + body
331     log_respuesta(con.puerto, f"{codigo} {mensaje}")
332     selector.modify(con.sock, selectors.EVENT_WRITE, data=con)
```

Imagen 15. Función enviar_error(con: Conexion, codigo: int, mensaje: str).

Descripción general: Se encarga de enviar los códigos de error, como los son el 400, 404, 405 entre otros.

Parámetros:

- **con : Conexión**
  - Objeto de conexión activa
- **codigo : int**
  - Código HTTP
- **mensaje : str**
  - Mensaje de estatus

- **escribir(con: Conexion)**

```
349 def escribir(con: Conexion) -> None:
350     enviados = con.sock.send(con.outb)
351     con.outb = con.outb[enviados:]
352     if not con.outb:
353         time.sleep(3) # simula cliente activo
354         cerrar(con)
```

Imagen 16. Función escribir(con: Conexion).

Descripción general: Se encarga de enviar el contenido del buffer de salida en bloques, permite escritura no bloqueante y cierra la conexión al terminar.

Parámetros:

- **con : Conexion**
  - Objeto de conexión activa

- **cerrar(con: Conexion)**

```
366  def cerrar(con: Conexion) ->None:
367      selector.unregister(con.sock)
368      con.sock.close()
369      conexiones_activas[con.puerto] -= 1
```

Imagen 17. Función cerrar(con: Conexion).

Descripción general: Se encarga de eliminar el socket del selector, cerrar la conexión y de decrementar el contador del pool.

Parámetros:

- **con : Conexion**
  - Objeto de conexión activa

- **iniciar_servidor(puerto: int)**

```
383  def iniciar_servidor(puerto: int) -> None:
384      sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
385      sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
386      sock.bind((HOST, puerto))
387      sock.listen()
388      sock.setblocking(False)
389
390      selector.register(sock, selectors.EVENT_READ, data=puerto)
391      print(f"Servidor escuchando en http://{HOST}:{puerto}")
```

Imagen 18. iniciar_servidor(puerto: int).

Descripción general: Se encarga de crear el socket TCP, lo pone en modo no bloqueante y lo registra en el selector.

Parámetros:

- **puerto : int**
  - Puerto del servidor principal

- **main()**

```python
def main() -> None:
    iniciar_servidor(PORT_PRIMARIO)
    iniciar_servidor(PORT_SECUNDARIO)

    while True:
        for key, mask in selector.select():
            if isinstance(key.data, int):
                aceptar(key.fileobj, key.data) # type: ignore
            else:
                con = key.data
                if mask & selectors.EVENT_READ:
                    leer(con)
                if mask & selectors.EVENT_WRITE:
                    escribir(con)

if __name__ == "__main__":
    main()
```

Imagen 19. Función main().

Descripción general: Se encarga de inicializar los servidores en el puerto 8080 y 8081 respectivamente.

CONCLUSIÓN

- **García Escamilla Bryan Alexis**

A diferencia de la práctica anterior, esta usa sockets de flujo no bloqueante, esto quiere decir que ahora el servidor en lugar de esperar a que llegue un cliente, a que lleguen datos y a que se pueda enviar información, ahora lo que hace es preguntarse si ese socket está listo para leer o escribir. Para ello se usa **selectors**, un objeto que observa a muchos sockets y le avisa al servidor cuando un socket puede leer y escribir, esto permite atender a muchos clientes simultáneamente, nunca espera y todo se hace en un solo hilo.

Como un agregado a la práctica, ahora el servidor va a determinar cuándo un recurso se debe descargar o mostrar en el navegador, para ello se usan los mismos tipos de MIME que ahora se encargan de indicar que recurso deberá mostrar el navegador, si un recurso no se encuentra entre ellos entonces el servidor fuerza la descarga automática enviando en la respuesta **Content-Disposition: attachment**, indicando la descargar, entre estos se pueden encontrar archivos .exe, .bin entre otros.

Para el caso de las carpetas, la descarga también es automática, pero en este caso la carpeta y todo su contenido se comprime en un ZIP y se descargar. Si la ruta de la carpeta no termina en '/' entonces el servidor se encarga de redirigir a la ruta correcta para posteriormente descargarla.

Y por último para la parte del pool de conexiones, especialmente fue complicado probarlo, puesto que la conexiones se cerraban instantáneamente por lo que se tuvo que hacer que estas permanecieran 5 segundos activas, mientras que el número máximo de conexiones activas se tuvo que reducir a dos para que se pudiera observar la redirección al segundo servidor.

- **Meléndez Macedonio Rodrigo**

Esta práctica al ser una continuación de la anterior, digamos que fue más fácil de llevar a cabo, pues los conceptos que se usan en esta son casi los mismos, sin embargo, la diferencia más notable en esta práctica es el uso de sockets no bloqueantes, los cuales esta constantemente operando, esperando recibir determinada información, ya sea para lectura o escritura. En este caso, usamos los sockets no bloqueantes en conjunto con una biblioteca de Python, llamada selectors. Esta biblioteca nos ayudó a tener un mejor control de los sockets no bloqueantes, pues nos ayudó a que el servidor identificará los sockets que estaban listos para leer o escribir, agilizando la atención a diversos clientes en simultaneo.

La otra parte de la práctica consistía en implementar una petición que permitiera la descarga de archivos desde un servidor HTTP de acuerdo con la ruta construida. Aquí, el cliente puede descargar el archivo haciendo clic en el vínculo y aquí es importante mencionar que, en caso de que la ruta no tenga la última diagonal "/" al final de esta, el servidor se encargará de construir la ruta completa para comenzar con la descarga, pero antes de eso, el contenido de la carpeta es convertido a un archivo ZIP para su optima descarga.

Adicional a esto, el servidor HTTP también contiene los métodos vistos en la anterior práctica, como es el caso de GET, para obtener datos o en este caso un archivo txt, POST para escribir sobre ese archivo txt, así como para realizar modificaciones de texto sobre este.